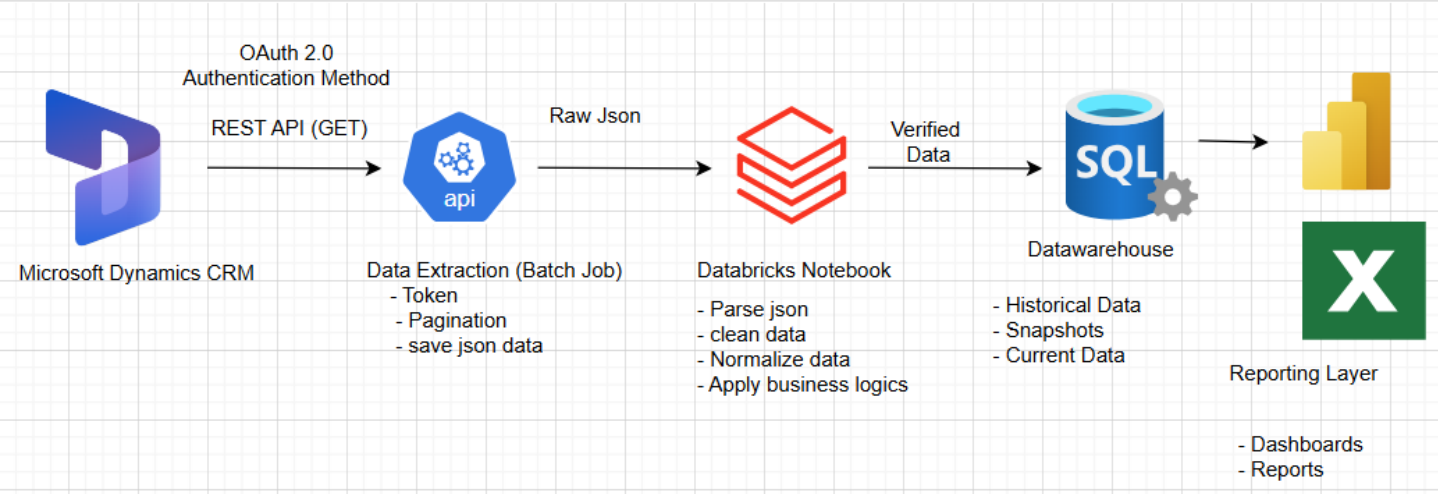
Extracting data from Microsoft Dynamics CRM

Stakeholders: Business Analysts, Business Development Team, leadership team members

Purpose: The main purpose of this project is to address some of the major challenges of construction-based management companies that is identifying and tracking current whereabouts of the project lifecycle. A construction project goes through several different phases and lots of changes occurs during its entire lifecycle, keeping track of all those changes and updating the respective stakeholders about those changes in a near real time is very critical in the construction-based management companies.

High-level Architecture Diagram of the Technical Workflow



Technical Summary:

Cadence: This data architecture pipeline will run every hour during the office hours through 8:00 A:M EST to 5:00 P:M EST during the Weekdays.

Microsoft Dynamics CRM: Microsoft Dynamics CRM is a platform that consists of five different customer relationships management solutions in a single location. CRM offers software for sales, marketing, customer service, field service, and project operations prioritizing the relationship building goal with its customers. Microsoft CRM is a very user-friendly and easy to use application. It helps the users and developers to organize and manage the customer related information is a single platform which makes the communication relationship easier with the customer and offers the best user experience as well.

API: To extract data using API, Microsoft's official documentations are being used. To obtain the bearer token using the OAuth authentication method the official documentation Link has been used.

App Registration

App Registration is required to register the application in the Microsoft Entra ID tenant. Next step is to grant access to the Dataverse for the App registration. The permission required is delegated permission.

Sample Code to connect using application secret

```
string SecretID = "00000000-0000-0000-0000-000000000000";
string AppID = "00001111-aaaa-2222-bbbb-3333cccc4444";
string InstanceUri = "https://yourorg.crm.dynamics.com";

string ConnectionStr = $@"AuthType=ClientSecret;
                        SkipDiscovery=true;url={InstanceUri};
                        Secret={SecretID};
                        ClientId={AppID};
                        RequireNewInstance=true";
using (ServiceClient svc = new ServiceClient(ConnectionStr))
{
    if (svc.IsReady)
    {
    //your code goes here
    }

}
```

Using OData to query the Microsoft CRM Dynamics Data:

Documentation Link :

To obtain the data, we need to use GET method:

Pseudo Code:

Request:

```
GET [Organization URI]/api/data/v9.2/
Accept: application/json
OData-MaxVersion: 4.0
OData-Version: 4.0
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json; odata.metadata=minimal
OData-Version: 4.0

{
    "@odata.context": "[Organization URI]/api/data/v9.2/$metadata",
    "value": [
        {
            "name": "aadusers",
            "kind": "EntitySet",
            "url": "aadusers"
        },
        {
            "name": "accountleadscollection",
            "kind": "EntitySet",
            "url": "accountleadscollection"
        },
        {
            "name": "accounts",
            "kind": "EntitySet",
            "url": "accounts"
        },

   [
}
```

To extract data from entity accounts:

```
GET [Organization URI]/api/data/v9.2/accounts
```

Retrieving Data:

To retrieve the data from a collection, we need to send a GET requests to the collection resource.

Request:

```
GET [Organization
URI]/api/data/v9.2/accounts?$select=name,statecode,statuscode&$orderby=name&$top=
1
Accept: application/json
OData-MaxVersion: 4.0
OData-Version: 4.0
Prefer: odata.include-annotations="OData.Community.Display.V1.FormattedValue"
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json; odata.metadata=minimal
OData-Version: 4.0
Preference-Applied: odata.include-
annotations="OData.Community.Display.V1.FormattedValue"

{
   "@odata.context": "[Organization
URI]/api/data/v9.2/$metadata#accounts(name,statecode,statuscode)",
   "value": [
      {
         "@odata.etag": "W/\"112430907\"",
         "name": "A. Datum Corporation (sample)",
         "statecode@OData.Community.Display.V1.FormattedValue": "Active",
         "statecode": 0,
         "statuscode@OData.Community.Display.V1.FormattedValue": "Active",
         "statuscode": 1,
         "accountid": "4b757ff7-9c85-ee11-8179-000d3a9933c9"
      }
   ]
}
```

Pagination:

There were more than 500 records in the data source. The limit is 500, so we need to apply the pagination concept to obtain all the data using @odata.nextLink annotation.

Request:

```
GET [Organization
URI]/api/data/v9.2/contacts?$select=fullname&$skiptoken=%3Ccookie%20pagenumber=%2
22%22%20pagingcookie=%22%253ccookie%2520page%253d%25221%2522%253e%253ccontactid%2
520last%253d%2522%257bD5026A4D-D01C-ED11-B83E-
000D3A572421%257d%2522%2520first%253d%2522%257b49B0BE2E-D01C-ED11-B83E-
000D3A572421%257d%2522%2520%252f%253e%253c%252fcookie%253e%22%20istracking=%22Fal
se%22%20/%3E
Accept: application/json
OData-MaxVersion: 4.0
OData-Version: 4.0
Prefer: odata.maxpagesize=2
```

Response:

```
HTTP/1.1 200 OK
OData-Version: 4.0
Preference-Applied: odata.maxpagesize=2

{
    "@odata.context": "[Organization
URI]/api/data/v9.2/$metadata#contacts(fullname)",
    "value": [
        {
            "@odata.etag": "W/\"80648710\"",
            "fullname": "Nancy Anderson (sample)",
            "contactid": "72bf4d48-34cb-ed11-b596-0022481d68cd"
        },
        {
            "@odata.etag": "W/\"80648724\"",
            "fullname": "Maria Campbell (sample)",
            "contactid": "74bf4d48-34cb-ed11-b596-0022481d68cd"
        }
    ],
    "@odata.nextLink": "[Organization
URI]/api/data/v9.2/contacts?$select=fullname&$skiptoken=%3Ccookie%20pagenumber=%2
23%22%20pagingcookie=%22%253ccookie%2520page%253d%25222%2522%253e%253ccontactid%2
520last%253d%2522%257bF2318099-171F-ED11-B83E-
000D3A572421%257d%2522%2520first%253d%2522%257bBB55F942-161F-ED11-B83E-
000D3A572421%257d%2522%2520%252f%253e%253c%252fcookie%253e%22%20istracking=%22Fal
se%22%20/%3E"
}
```

Databricks Notebook

Databricks Notebook is used to read the json data that is obtained from the web api using the GET method. In the databricks notebook various python and pyspark codes are used to convert the json into tabular data, clean the data, apply business logic and then validate the data. Once the data is validated then it is loaded into the Azure Sql database.

Sample code using DUMMY DATA SETS

```
import os
import time
import json
import requests
from datetime import datetime, timezone
import msal
```

```
# -------------
# CONFIG
# -------------
TENANT_ID        = os.getenv("TENANT_ID", "YOUR_TENANT_ID")
CLIENT_ID        = os.getenv("CLIENT_ID", "YOUR_APP_CLIENT_ID")
CLIENT_SECRET    = os.getenv("CLIENT_SECRET", "YOUR_APP_CLIENT_SECRET")

# Your Dataverse / Dynamics environment base URL (example):
# https://yourorg.crm.dynamics.com
CRM_RESOURCE     = os.getenv("CRM_RESOURCE", "https://yourorg.crm.dynamics.com")

# Web API version commonly seen as v9.2 in docs/examples for Dataverse Web API
API_VERSION      = os.getenv("API_VERSION", "v9.2")

# Entity set name examples:
# projects might be "msdyn_projects" depending on your schema
ENTITY_SET       = os.getenv("ENTITY_SET", "msdyn_projects")

# Incremental watermark storage
WATERMARK_FILE   = os.getenv("WATERMARK_FILE",
"/tmp/watermark_msdyn_projects.txt")


# ----------------------------
# AUTH: OAuth token acquisition (client credentials)
# OAuth is required and MSAL is recommended for Dataverse access
# ----------------------------
def get_access_token():
    authority = f"https://login.microsoftonline.com/{TENANT_ID}"
    app = msal.ConfidentialClientApplication(
        CLIENT_ID,
        authority=authority,
        client_credential=CLIENT_SECRET
    )

    # For Dataverse, you typically request {resource}/.default
    scope = [f"{CRM_RESOURCE}/.default"]
    result = app.acquire_token_for_client(scopes=scope)

    if "access_token" not in result:
        raise RuntimeError(f"Token acquisition failed: {result}")
    return result["access_token"]


# ----------------------------
```

```
# WATERMARK: last successful modified timestamp
# ----------------------------
def read_watermark():
    if not os.path.exists(WATERMARK_FILE):
        # default old date
        return "2000-01-01T00:00:00Z"
    with open(WATERMARK_FILE, "r") as f:
        return f.read().strip()

def write_watermark(value_iso):
    with open(WATERMARK_FILE, "w") as f:
        f.write(value_iso)


# ----------------------------
# EXTRACT: OData query + paging
# Dataverse paging: use Prefer: odata.maxpagesize and follow @odata.nextLink
# ----------------------------
def fetch_entity_incremental(access_token, modified_since_iso, page_size=5000):
    headers = {
        "Authorization": f"Bearer {access_token}",
        "Accept": "application/json",
        "OData-MaxVersion": "4.0",
        "OData-Version": "4.0",
        # Page control per Microsoft guidance 【5-d23508】
        "Prefer": f"odata.maxpagesize={page_size}"
    }

    # Minimal example: select fields + modifiedon watermark filter
    # Your real field names will differ.
    select_cols =
"msdyn_projectid,msdyn_subject,modifiedon,msdyn_startdate,msdyn_enddate,msdyn_bud
getamount,statuscode"

    # OData filter: "modifiedon gt <timestamp>"
    # (Exact column names depend on entity schema.)
    url = (
        f"{CRM_RESOURCE}/api/data/{API_VERSION}/{ENTITY_SET}"
        f"?$select={select_cols}"
        f"&$filter=modifiedon gt {modified_since_iso}"
    )

    all_rows = []
    next_url = url
```

```
    while next_url:
        resp = requests.get(next_url, headers=headers, timeout=60)
        if resp.status_code == 401:
            # token might have expired; refresh
            access_token = get_access_token()
            headers["Authorization"] = f"Bearer {access_token}"
            resp = requests.get(next_url, headers=headers, timeout=60)

        resp.raise_for_status()
        payload = resp.json()

        rows = payload.get("value", [])
        all_rows.extend(rows)

        # Paging: @odata.nextLink indicates next page
        next_url = payload.get("@odata.nextLink")

    return all_rows


def main():
    token = get_access_token()
    watermark = read_watermark()

    rows = fetch_entity_incremental(token, watermark, page_size=2000)

    # Update watermark to now after successful extract
    new_watermark = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
    write_watermark(new_watermark)

    # Write raw extract to disk (or ADLS/S3 in real system)
    raw_path = "/tmp/msdyn_projects_raw.json"
    with open(raw_path, "w") as f:
        json.dump(rows, f)

    print(f"Extracted {len(rows)} rows into {raw_path}")
    print(f"Updated watermark => {new_watermark}")


if __name__ == "__main__":
    main()
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, lit, when, to_date, to_timestamp, current_timestamp,
    regexp_replace, upper, coalesce, row_number
)
from pyspark.sql.window import Window

spark = SparkSession.builder.appName("CRM_ETL").getOrCreate()

# ----------------------------
# 1) READ RAW JSON
# ----------------------------
raw_path = "/tmp/msdyn_projects_raw.json"
df_raw = spark.read.json(raw_path)

# -------------------
# 2) DATA CLEANING
# --------------------
df_clean = (
    df_raw
    # Standardize string fields
    .withColumn("project_name", upper(regexp_replace(col("msdyn_subject"),
r"\s+", " ")))
    # Convert dates/timestamps (field names are examples)
    .withColumn("modifiedon_ts", to_timestamp(col("modifiedon")))
    .withColumn("start_date", to_date(col("msdyn_startdate")))
    .withColumn("end_date", to_date(col("msdyn_enddate")))
    # Ensure numeric budget exists
    .withColumn("budget_amount", col("msdyn_budgetamount").cast("double"))
    # Fill missing values with safe defaults
    .withColumn("budget_amount", coalesce(col("budget_amount"), lit(0.0)))
    .drop("msdyn_subject", "msdyn_startdate", "msdyn_enddate",
"msdyn_budgetamount")
)

# --------------------
# 3) BUSINESS LOGIC
# ---------------------

# Example 1: Project health based on date and budget
df_logic = (
```

```
    df_clean
    .withColumn(
        "project_timing_flag",
        when(col("end_date").isNull(), lit("UNKNOWN"))
        .when(col("end_date") < to_date(current_timestamp()), lit("PAST_DUE"))
        .otherwise(lit("ON_TRACK"))
    )
    .withColumn(
        "budget_flag",
        when(col("budget_amount") >= 10_000_000, lit("HIGH"))
        .when(col("budget_amount") >= 1_000_000, lit("MEDIUM"))
        .otherwise(lit("LOW"))
    )
    # Example 2: Normalize status codes
    .withColumn(
        "project_status",
        when(col("statuscode") == 1, lit("ACTIVE"))
        .when(col("statuscode") == 2, lit("ON_HOLD"))
        .when(col("statuscode") == 3, lit("CLOSED"))
        .otherwise(lit("OTHER"))
    )
    .withColumn("etl_loaded_at", current_timestamp())
)

# ----------------------------
# 4) DEDUPLICATION / LATEST RECORD PER PROJECT
# ----------------------------
# Choose "latest modifiedon" per msdyn_projectid
w =
Window.partitionBy(col("msdyn_projectid")).orderBy(col("modifiedon_ts").desc())
df_dedup = (
    df_logic
    .withColumn("rn", row_number().over(w))
    .filter(col("rn") == 1)
    .drop("rn")
)

# ----------------------------
# 5) QUALITY CHECKS
# ----------------------------



df_final = (
    df_dedup
```

```
    .filter(col("msdyn_projectid").isNotNull())
    .dropDuplicates(["msdyn_projectid"])
)

# ------------------------------
# 6) WRITE TO AZURE SQL DATABASE
# ------------------------------
jdbcHostname = "yourserver.database.windows.net"
jdbcPort = 1433
jdbcDatabase = "yourdb"
jdbcUsername = "youruser"
jdbcPassword = "yourpassword"

jdbcUrl = (
    f"jdbc:sqlserver://{jdbcHostname}:{jdbcPort};"
    f"database={jdbcDatabase};"
    "encrypt=true;trustServerCertificate=false;"
    "hostNameInCertificate=*.database.windows.net;"
    "loginTimeout=30;"
)

target_table = "dbo.crm_projects_curated"

connectionProperties = {
    "user": jdbcUsername,
    "password": jdbcPassword,
    "driver": "com.microsoft.sqlserver.jdbc.SQLServerDriver"
}

(
    df_final.write
    .mode("append")   # or "overwrite" for full refresh
    .jdbc(url=jdbcUrl, table=target_table, properties=connectionProperties)
)

print(f"Wrote {df_final.count()} records to {target_table}")
```

Datawarehouse: All the verified data is now loaded in the Azure Sql database. Azure Sql database works as a Datawarehouse where all the historical, current and snapshot data is stored for all the reporting purposes.

Reporting Layer: Reporting tools such as Power BI and Microsoft Excel are used to create reports and dashboards using the data stored in the tables of the Azure Sql database.

References:

- https://learn.microsoft.com/en-us/power-apps/developer/data-platform/authenticate-oauth
- https://learn.microsoft.com/en-us/power-apps/developer/data-platform/webapi/query/overview
- https://learn.microsoft.com/en-us/power-apps/developer/data-platform/webapi/query/page-results